Forense computacional em discos de estado sólido: desafios e perspectivas

João Vitor A. Ribeiro[1], **João Gondim**[1]

[1]Departamento de Ciência da Computação – Universidade de Brasília (UnB)
Campus Darcy Ribeiro Edifício CIC/EST Asa Norte Caixa Postal 4466

jaavrr@gmail.com,gondim@unb.br

Abstract. *An analysis of the usability and the effectiveness of the standard practices in Data Recovery and Digital Forensics in a SSD - solid state disk using common data and file recovery tools and a black-box methodology is presented. The process of recovering data from a SSD using common tools of Data and File Recovery, Data Carving and with background knowledge in File Systems and in SSD technology, provided the basis for a comparative analysis with the results found in the bibliography. Results suggest that the technology used in SSDs can hinder or even prevent Forensic Analysis on Linux and Windows systems.*

Resumo. *Este trabalho apresenta uma análise da usabilidade e da eficácia de técnicas usuais de recuperação de dados e de forense digital em um SSD - disco de estado sólido utilizando ferramentas usuais de recuperação de dados e uma metodologia black-box. É feito um estudo de caso em um SSD utilizando ferramentas usuais de recuperação de dados, data carving e informações pertinentes a tecnologia dos dispositivos e sistemas de arquivos. Também é realizada uma análise comparativa com os resultados da bibliografia. Os resultados obtidos sugerem que as tecnologias utilizadas em SSDs podem dificultar ou até inviabilizar a análise forense nos Sistemas Operacionais Windows e Linux.*

1. Introdução

Nos últimos tempos, tem-se verificado um crescimento no uso dos discos de estado sólido - SSD *Solid State Disk* em detrimento dos discos rígidos (*Hard Disk Drive*). No entanto, os SSDs apresentam diversos desafios à forense computacional em comparação com os discos rígidos tradicionais. Entre eles, mecanismos internos que podem dificultar ou inviabilizar a recuperação de dados excluídos nessas mídias, podendo levar à perda de informação que pode constituir potenciais evidências.

São dois os principais fatores que podem levar a essa perda: o processo de coleta de lixo e o comando TRIM. Isso se deve em parte por conta do fato de os SSDs poderem fazer uso desses mecanismos de forma autônoma, ou seja, sem a necessidade da ação de um Sistema Operacional (SO) ou do usuário, sendo também possível que esses mecanismos sejam acionados pelos agentes citados. [Bell and Boddington 2010].

O trabalho se constitui num estudo de habilidade da aplicação das técnicas e ferramentas forenses abertas utilizadas em HDD's quando aplicadas a SSD's. Esse estudo levou a uma análise comparativa com os resultados mais recentes da área.

 ©2016 SBC — Soc. Bras. de Computação

2. Discos de Estado Sólido

Nos últimos anos, os SSDs têm se tornado uma tecnologia emergente. SSDs são baseados em circuitos integrados, fornecendo diversas vantagens como menor consumo de energia, menor tamanho, resistência a choques e a característica mais impotante de alto desempenho no acesso a dados aleatórios [Chen et al. 2009].

A maioria dos SSDs atuais utiliza a arquitetura NAND planar, ou seja em duas dimensões. Porém já se encontram no mercado dispositivos com a tecnologia V-NAND (*Vertical* NAND) na qual tem-se o empilhamento de camadas de células na vertical, proporcionando maior densidade e capacidade de armazenamento [Samsung 2016].

2.1. Arquitetura de um SSD

Os SSDs são normalmente construídos utilizando arranjos de memória *Flash*, que são ligados por um barramento serial a uma **controladora** [Chen et al. 2009]. A controladora é um dos elementos mais importantes de um SSD, pois é responsável pela execução de rotinas e controle de mecanismos internos. A controladora recebe e processa requisições do Sistema de Arquivos por meio de uma interface de conexão como, por exemplo, a interface **SATA -** *Serial AT Attachment*.

É possível pensar a arquitetura de um SSD em três camadas (abstrações). São elas: HIL - *Host Interface Layer*, FTL - *Flash Translation Layer* e FIL - *Flash Interface Layer*. A Figura 1 ilustra essa divisão.

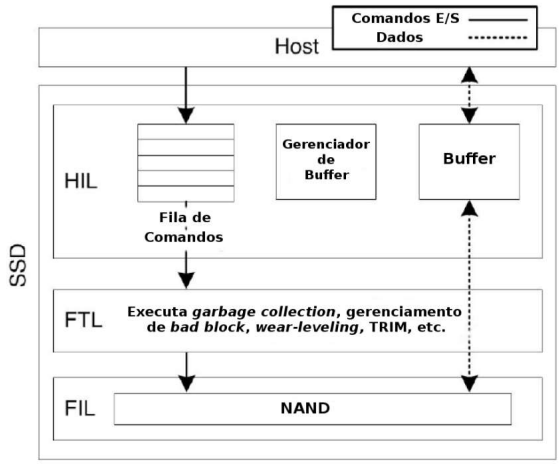


Figura 1. Arquitetura de um SSD
[Junior and Queiroz 2015b]

Na *Host Interface Layer*, comandos de E/S (Entrada/Saída) fornecidos pelo Sistema de Arquivos e/ou SO são enfileirados e atendidos na ordem em que chegam. Na *Flash Translation Layer* ocorre gerência de certos mecanismos internos utilizados para otimizar o desempenho e por emular o funcionamento de um HDD em termos lógicos, possibilitando a compatibilidade e a interoperabilidade entre os sistemas computacionais. Como sumariza [Chen et al. 2009], essa camada desempenha três papeis fundamentais: **Mapeamento lógico de blocos, Nívelamento de desgaste e Coleta de lixo**.

 ©2016 SBC — Soc. Bras. de Computação

Por fim a *Flash Interface Layer*, camada de nível mais baixo, é constituída em essência pelas memórias NAND. Nela, são realizadas as operações de leitura, escrita e apagamento. Sendo que na leitura a menor unidade é a página. A escrita também é geralmente realizada em termos de páginas, e o apagamento é feito em nível de blocos [Chen et al. 2009].

2.2. Mapeamento lógico de blocos

Os discos de estado sólido operam tipicamente pelo armazenamento de dados em blocos de 512 kB, subdivididos em páginas de 4 kb, resultando em 128 páginas por bloco nessa configuração [Olson and Langlois 2008]. Tais blocos são constituídos de arranjos de transístores *NAND* semelhantes aos chips utilizados em processadores.

2.3. Nivelamento de desgaste

Geralmente as operações de escrita apresentam certa localidade, ou seja, frequentemente as mesmas regiões de armazenamento são escritas (princípio da localidade espacial). Naturalmente, esse princípio se aplica às requisições de blocos do SSD, onde um conjunto de blocos (lógicos) é constantemente referenciado para escrita pelo Sistema de Arquivos.

Como as memórias *Flash* possuem um número limitado de ciclos *erase-write*, foi concebido o mecanismo de nivelamento de desgaste (*wear leveling*) que uniformiza essas operações pelos blocos físicos do disco. Logo, escritas subsequentes em um bloco lógico na realidade correspondem a escritas em blocos diferentes (a controladora copia o conteúdo de um bloco físico para outro de modo a evitar que o primeiro seja sobrescrito). A Figura 2 ilustra a atuação do nivelamento de desgaste nas células de um SSD.

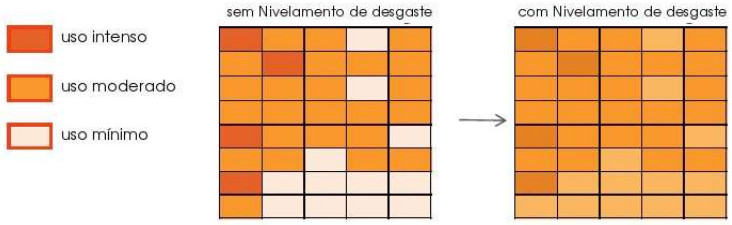


Figura 2. Nivelamento de desgaste
Adaptada de [Nazarian and Dubois 2013]

2.4. Coleta de lixo

Os dados são escritos na granularidade de páginas, porém em geral apenas blocos inteiros podem ser apagados [Bell and Boddington 2010]. À medida em que o SSD vai sendo utilizado, restam menos blocos sem páginas livres por conta do nivelamento de desgaste.

Dessa forma, quando se deseja reutilizar uma página que contém dados desatualizados, é necessário copiar as páginas válidas desse bloco para outro bloco na mídia, para que o primeiro seja apagado. Esse processo é custoso e é um dos grandes limitadores de desempenho no longo prazo.

Para amenizar esse problema, foram concebidas rotinas para a controladora do SSD que identificam blocos contendo páginas não utilizadas ou contendo dados desatualizados, antes de uma operação de escrita. Após a identificação, a controladora realiza

 ©2016 SBC — Soc. Bras. de Computação

o procedimento explicado acima sempre que possível. Essa rotina recebe o nome de ”coleta de lixo”, porém na verdade a rotina faz a ”coleta” tanto de dados dados válidos quanto inválidos. Entretanto há um problema central pois a controladora do SSD ”não sabe” o que são dados inválidos (ou lixo), já que essa informação é dependente do Sistema de Arquivos e do SO e cada combinação desses define um conjunto de regras para o remanejo dos blocos. O SO não comunica à controladora os blocos passíveis de apagamento. [Bell and Boddington 2010] conjecturam que após a cópia de uma página pelo mecanismo do *wear leveling*, esta está sujeita a coleta de lixo em um momento oportuno. Nesse cenário, seria possível o apagamento automático sem a comunicação do SO.

Existem, portanto, dois tipos principais de coleta de lixo: a *background garbage collection* e a *filesystem-aware garbage collection*. A primeira trata do caso mais geral, onde foi enviada uma requisição de escrita a um bloco em uso. Já a segunda se refere à coleta de lixo onde a controladora ”ciente” do Sistema de Arquivos em uso, verifica o histórico de arquivos apagados e informações globais sobre o Sistema de Arquivos. Para lidar com o cenário mais geral, foi criado o comando TRIM que será abordado na Seção 3.

2.5. Provisionamento

Nos SSDs, o provisionamento é um mecanismo que objetiva aumentar o tempo de vida do dispositivo, atuando junto com o nivelamento de desgaste e com a coleta de lixo. Algumas áreas (blocos) são reservados para a controladora, sendo estas chamadas de área de sobre-provisionamento ou em inglês, *over-provisioning area* (OP). Esses blocos podem ser utilizados quando uma operação de escrita é enviada, requisitando bloco(s) para escrita. Esta é realizada na OP, enquanto que um bloco *dirty* (inválido para escrita, ou apagado) entra na área de provisionamento. Os blocos *dirty* são apagados (coleta de lixo) quando o SSD encontra-se em estado de baixa atividade [Gubanov and Afonin 2014].

Por conta do provisionamento, os SSDs possuem uma quantidade de memória física maior do que o especificado (ou reconhecido pelo SO), porém a memória efetiva continua a mesma.

3. Comando TRIM

O comando TRIM (em português aparar ou podar) é um comando no nível de SO que sinaliza à controladora que existem blocos não utilizados (que foram excluídos) para que esta apague previamente os blocos de modo a antecipar posteriores escritas, pois estas requerem um apagamento prévio. A Figura 3 ilustra o funcionamento do TRIM.

Cada SSD implementa de forma diferente o comando TRIM. Alguns implementam o que é chamado **DRAT** - *Deterministic Read After Trim* (Leitura Determinística Após o Trim) ou **DZAT** - *Deterministic Zeroes After Trim* - (”Zeros” Determinísticos Após o Trim) [Gubanov and Afonin 2014]. No primeiro, após o envio do comando TRIM pelo SO à controladora seguido de um comando de leitura desses blocos, o SSD retorna blocos contendo valores fixos. Já no segundo, esses valores são 0s. Outra implementação possível do comando retorna o conteúdo dos blocos até que as rotinas internas de coleta de lixo entrem em ação.

Cada fabricante define como a controladora implementa o comando TRIM, pois não há um padrão. Dessa forma, no caso de um certo par SO e Sistema de Arquivos que

 ©2016 SBC — Soc. Bras. de Computação

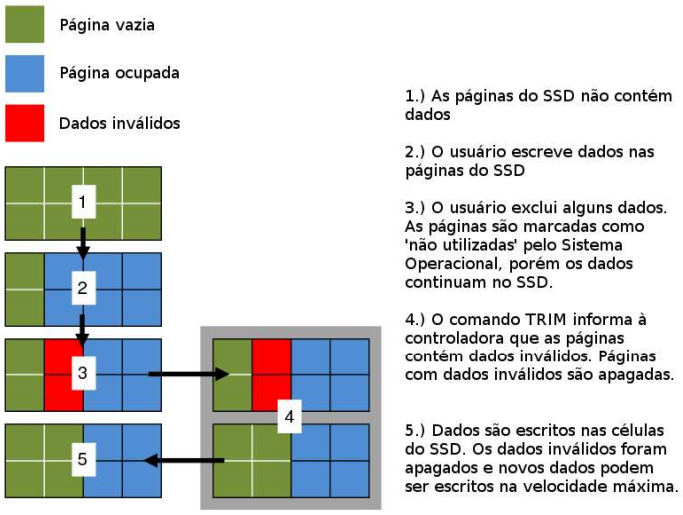


Figura 3. Funcionamento do comando TRIM
Adaptada de [Corsair 2010]

suporte o comando TRIM, o processo de exclusão e posterior leitura dos blocos lógicos "podados" poderá apresentar diferentes comportamentos quando comparado a outro par SO e Sistema de Arquivos. Em alguns sistemas operacionais, o TRIM é enviado automaticamente. Em outros, é dependente de configuração, podendo ser necessária uma demanda explícita.

3.1. Suporte ao TRIM nos Sistemas Operacionais

Entre os sistemas operacionais que oferecem suporte ao comando TRIM encontram-se: Windows 7 e sucessores [Microsoft 2008] , Mac OS X (a partir da versão 10.6.8) [Slivka 2011] e o Linux [KernelNewbies 2008]. O suporte não é pleno, pois depende também do Sistema de Arquivos e poderá depender de habilitação ou configuração prévia.

4. Trabalhos relacionados

Antonellis [Antonellis 2008] realiza um experimento de exclusão de imagens e de um arquivo de texto, sob o ambiente *Windows* com o Sistema de Arquivos **NTFS** obtendo apenas arquivos com 0s. O autor não menciona a presença ou a influência do comando TRIM, assim como também não menciona a marca e o modelo do SSD.

Bell e Boddington [Bell and Boddington 2010] realizam experimentos de recuperabilidade no sistema *Windows XP* que não possui suporte ao comando TRIM, obtendo resultados ínfimos para a recuperação dos arquivos.

Gebremaryam [Gebremaryam 2011] realiza experimentos de recuperabilidade no sistema *Windows 7* na presença do TRIM, não obtendo sucesso. Já os experimentos na ausência do TRIM foram bem sucedidos.

King e Vidas [King and Vidas 2011] realizam experimentos nos sistemas *Windows* e *Linux* para uma vasta gama de SSDs obtendo resultados variados.

Bonetti et al. [Bonetti et al. 2011] realizam experimentos nos sistemas *Windows*

 ©2016 SBC — Soc. Bras. de Computação

e *Linux*, na presença do TRIM, não obtendo sucesso. Já os experimentos na ausência do TRIM foram bem sucedidos.

Nisbet et al. [Nisbet et al. 2013] realizam experimentos nos sistemas *Windows*, *Linux* e no *Mac OS X* com resultados variados. Quanto ao uso do TRIM como uma medida anti-forense, os autores concluem que o TRIM por si só não é uma medida adequada para a sanitização de um SSD, relatando que em alguns casos, 185 a 214 MB remanesceram em um disco após a instrução do comando.

Júnior e Ruy [Junior and Queiroz 2015a] em 2015 afirmam que foi possível recuperar arquivos excluídos em um SSD da marca ADATA, modelo Premier SP800 de 32GB que possui suporte ao TRIM. Os autores não entram em detalhes sobre até que ponto se dá essa recuperabilidade nem sobre o procedimento utilizado.

5. Estudo de caso

A partir de um SSD da marca Samsung modelo 850 EVO, foi realizado um estudo de caso. O SSD em questão encontrava-se em estado seminovo, onde foi sujeito ao uso de um usuário comum, utilizando certas aplicações como navegador, *softwares* de edição de texto, terminal, visualizadores de mídia (aúdio, texto) etc. configurando-se, em maioria, uso de baixo impacto para o sistema de arquivos. A metodologia utilizada é a *black-box*. Dessa forma, não está no escopo deste trabalho avaliar a eficácia ou a usabilidade de ferramentas específicas. Também há que se mencionar que provavelmente há cache no dispositivo.

5.1. Cenários

Foram realizados cinco experimentos, com base em cinco cenários, com o intuito de comprovar ou contradizer os resultados atuais e realizar uma análise comparativa. Os cenários foram divididos em dois grupos: sistema *Linux* com Sistema de Arquivos **ext4** e sistema *Windows* com o Sistema de Arquivos **NTFS**.

No sistema *Linux*:

1. Exclusão de arquivos, sem a atuação do comando TRIM em qualquer forma e posterior tentativa de recuperação, com o objetivo de verificar a atuação da coleta de lixo;
2. Exclusão de arquivos, seguida do envio do comando TRIM manual após um tempo t e posterior tentativa de recuperação com o objetivo de verificar o efeito do comando TRIM e atuação da coleta de lixo.

Já no sistema *Windows*:

3. Exclusão de arquivos, sem a atuação do comando TRIM em qualquer forma e posterior tentativa de recuperação, com o objetivo de verificar a atuação da coleta de lixo;
4. Exclusão de arquivos, com o comando TRIM automático habilitado e posterior tentativa de recuperação, com o objetivo de verificar o efeito do comando TRIM quando enviado de forma autônoma pelo SO, além de verificar a atuação da coleta de lixo;
5. Exclusão de arquivos, seguida do envio do comando TRIM manual após um tempo t e posterior tentativa de recuperação com o objetivo de verificar o efeito do comando TRIM e atuação da coleta de lixo. O TRIM automático está desabilitado durante esse experimento.

 ©2016 SBC — Soc. Bras. de Computação

5.2. Arquivos

Foram selecionados oito arquivos de diferentes formatos conforme a Tabela 1:

Tabela 1. Arquivos selecionados

Nro	Nome	Tamanho	Tipo
1	(Aya Higuchi) Chopin - Waltz in A minor, B. 150.mp3	2.1 MB	Arquivo de música (.mp3)
2	campo01.mp4	35.6 MB	Vídeo (.mp4)
3	folhas01.png	28.8 kB	Imagem (.png)
4	milho01.jpg	933.5 kB	Imagem (.jpg)
5	terra01.gif	927.0 kB	Imagem (.gif)
6	texto01.txt	61 B	Texto (.txt)
7	texto02.rtf	120.0 kB	Texto (.rtf)
8	relatorio01.pdf	2.5 MB	Documento (.pdf)

Para cada caso de teste no sistema *Linux* foi criada uma imagem de partição contendo apenas 0s e esta foi montada. No caso do sistema *Windows* foi criada um volume para cada caso de teste, também formatado. Os arquivos selecionados foram copiados para os volumes e logo após excluídos. Após o decorrer de certo período de tempo após a exclusão, a imagem de disco no sistema *Linux* foi desmontada, e no sistema *Windows* foi criada a partir do volume. Sob esta imagem foram aplicadas algumas das ferramentas usuais de recuperação de dados constantes na Seção 5.5.

5.3. Criação das imagens

Para a criação das imagens de disco no sistema *Linux* foi utilizado o utilitário *dd* versão 8.23. As imagens possuiam 250000 blocos de tamanho 4096 *bytes* totalizando 1Gb de tamanho total e foram particionadas com o utilitário *fdisk* versão 2.25.2 e formatadas com o Sistema de Arquivos **ext4** utilizando o *mkfs.ext4*.

Os volumes criados no sistema *Windows* possuiam o mesmo formato de número de blocos e tamanho dos blocos, logo, mesmo tamanho de 1Gb. O volume foi formatado com o Sistema de Arquivos **NTFS**. As imagens dos volumes foram criadas utilizando o programa *ProDiscover Basic* versão 7.0.

5.4. Casos de teste

Para cada caso de teste foram realizados os seguintes procedimentos:

1. Criação da partição formatada contendo apenas 0s;
2. Cópia dos arquivos de teste para a partição;
3. Exclusão ”permanente”dos arquivos;
4. Envio do comando TRIM manualmente (Cenários 2 e 5);
5. Aguardam-se t segundos e a imagem de disco é criada (*Windows*) ou desmontada (*Linux*);
6. Tentativa de recuperação dos arquivos.

5.5. Recuperação

Sob as imagens forenses foram aplicadas as seguintes ferramentas:

No *Linux*:

 ©2016 SBC — Soc. Bras. de Computação

- *extundelete* versão 0.2.4 com a biblioteca libext2fs versão 1.42.12 por N E Case;
- *foremost* versão 1.5.7 por Jesse Kornblum, Kris Kendall, e Nick Mikus;
- *magicrescue* versão 1.1.9 por Jonas Jensen;
- *scalpel* versão 1.60 por Golden G. Richard III;
- *autopsy* versão 2.24 por Brian Carrier.

No *Windows*:

- *The Sleuth Kit* versão 4.2.0 por Brian Carrier;
- *FTK Imager* versão 3.4.2 por ACCESS DATA.

Para cada caso de teste, o objetivo era avaliar a viabilidade em recuperar os arquivos excluídos. Dessa forma, se uma ferramenta recuperou todos os arquivos de forma completa, não foram utilizadas as demais. Caso contrário, foram utilizadas todas as ferramentas sucessivamente. No escopo do trabalho não coube avaliar a eficácia das ferramentas na recuperação dos arquivos. Foi utilizado um comparador hexadecimal (*diff*) nos casos de recuperação parcial.

6. Resultados

No Cenário 1, no sistema *Linux* com o TRIM desabilitado, foi possível a recuperação total dos arquivos excluídos.

No Cenário 2, com o TRIM manual, foi possível recuperar os arquivos até um tempo t de 5s, que é a janela de oportunidade para a recuperação forense.

Já nos experimentos no sistema *Windows*, no Cenário 3, com o TRIM desabilitado, obteve-se a recuperação total dos arquivos, exceto um caso espúrio onde $t = 5$. Nesses caso, um dos arquivos encontrava-se com seu conteúdo "zerado"e o outro com conteúdo parcialmente "zerado". Levanta-se a hipótese de que o dado espúrio represente uma execução aparentemente aleatória das rotinas de coleta de lixo ou um erro na formação da imagem de disco ou algum outro evento não previsto, como a presença prévia do comando TRIM na controladora.

No Cenário 4, na presença do TRIM automático (agendado), obteve-se uma janela de oportunidade de 3s. Para $4s \leq t \leq 15s$, recuperação foi parcial. Para $t \geq 16s$, obteve-se a recuperação do arquivo de texto (arquivo número 6 na Tabela 1).

Por fim, no Cenário 5, na atuação do TRIM manual, verifcou-se uma janela de oportunidade menor que 1s. Para $t \leq 20s$, a recuperação foi parcial. Para $t \geq 20s$, obteve-se a recuperação do arquivo de texto (arquivo número 6 na Tabela 1).

7. Comparação com outros trabalhos

A Tabela 2 contém um resumo da análise comparativa com os resultados de outros trabalhos.

8. Conclusão e trabalhos futuros

Foram estudadas as características e mecanismos internos dos SSDs que podem inviabilizar ou dificultar a prática usual de recuperação forense e de forense computacional em dois Sistemas Operacionais, *Windows* e *Linux*.

 ©2016 SBC — Soc. Bras. de Computação

Tabela 2. Comparação com outros trabalhos

Autores	Resultados obtidos pelos autores	Comparação com o trabalho realizado
Bell & Boddington	Experimentos de recuperabilidade realizados no sistema *Windows XP* sem o TRIM, sem sucesso	Resultados obtidos pela dupla refutados pelos resultados dos experimentos dos Cenários 1 e 3
Gebremaryam	Experimentos no sistema *Windows 7* na presença (sem sucesso) e na ausência do TRIM (bem sucedidos)	Resultados se alinharam
King & Vidas	Experimentos nos sistemas *Windows* e *Linux*, vasta gama de SSDs com resultados variados	Confirmados para SSDs de algumas marcas e refutados por outros de outras
Bonetti et al.	Experimentos nos sistemas *Windows* e *Linux*, na presença do TRIM (sem sucesso) e na ausência do TRIM (bem sucedidos)	Resultados se alinharam, exceto para um modelo de SSD
Nisbet et al.	Experimentos nos sistemas *Windows*, *Linux* e no *Mac OS X* com resultados variados	Não coube comparação ($1hr \leq t \leq 5hrs$)

Os resultados foram variados, sendo que o fator determinante em ambos os sistemas é o comando TRIM. No sistema *Linux* a atuação do TRIM mostrou-se agressiva, inviablizando a recuperação dos arquivos após 5s. No sistema *Windows*, tanto a execução automática quanto a manual do comando TRIM apresentaram resultados negativos para a recuperação forense, à exceção de um arquivo de texto de tamanho na faixa dos *bytes*.

Em trabalhos futuros poderá ser estudada como se dá a interação do SO com a controladora do SSD nos instantes que se seguem após o envio do comando. Dessa forma poderão ser estudadas formas de impedir o envio do comando, retirá-lo da fila de comandos, impedir a sua execução, anular ou amenizar seus efeitos.

Trabalhos futuros também poderão ser direcionados no estudo das diferenças entre as implementações do TRIM em cada sistema bem como o estudo da granularidade dos arquivos e seus efeitos na recuperação forense, no sistema *Windows* com Sistema de Arquivos **NTFS**, dado que nesse sistema foi possível recuperar, em todos os testes, um arquivo com tamanho na faixa dos *bytes*.

Referências

Antonellis, C. J. (2008). *Solid State Disks and Computer Forensics* issa journal, pgs 36-38.

Bell, G. B. and Boddington, R. (2010). *Solid state drives: The beginning of the end for current practice in digital forensic recovery? Journal of Digital Forensics, Security and Law, 5(3):1-20.*

 ©2016 SBC — Soc. Bras. de Computação

Bonetti, G., Viglione, M., Frossi, A., Maggi, F., and Zanero, S. (2011). *A Comprehensive Black-box Methodology for Testing the Forensic Characteristics of Solid-state Drives* journal of computer virology and hacking techniques.

Chen, F., Koufaty, D. A., and Zhang, X. (2009). *Understanding intrinsic characteristics and system implications of flash memory based solid state drives In ACM SIGMETRICS Performance Evaluation Review, volume 37, pages 181-192. ACM.*

Corsair, I. (2010). *How to Check That TRIM is active* `http://www.corsair.com/media/cms/manual/How_To_Check_That_TRIM_is_Active.pdf`.

Gebremaryam, F. Y. (2011). *Solid State Drive (SSD) Digital Forensics Construction* master of science in computing system engineering, politecnico di milano.

Gubanov, Y. and Afonin, O. (2014). *Recovering Evidence from SSD Drives in 2014: Understanding TRIM, Garbage Collection and Exclusions.*

Junior, M. A. C. C. and Queiroz, R. J. G. B. (2015a). *Apos um processo de formatacao, e possivel recuperar dados em um SSD?* universidade federal de pernambuco (ufpe).

Junior, M. A. C. C. and Queiroz, R. J. G. B. (2015b). *Forense em Solid State Drive: entendendo os mecanismos internos que podem inviabilizar a recuperacao de dados* universidade federal de pernambuco (ufpe).

KernelNewbies (2008). *Linux 2 6 28* `http://kernelnewbies.org/Linux_2_6_28\#head-a1a9591f48fe0cf8fde1f0d1f637c7ae54ad0bfa`.

King, C. and Vidas, T. (2011). *Empirical analysis of solid state disk data retention when used with contemporary operating systems* digital investigation journal.

Microsoft, C. (2008). *Windows 7 Enhancements for Solid-State Drives* `http://download.microsoft.com/download/F/A/7/FA70E919-8F82-4C4E-8D02-97DB3CF79AD5/COR-T558_Shu_Taiwan.pdf`.

Nazarian, H. and Dubois, S. (2013). *The drive for SSDs: What's holding back NAND flash?* `http://www.edn.com/design/systems-design/4424905/2/The-drive-for-SSDs--What-s-holding-back-NAND-flash--`.

Nisbet, A., Lawrence, S., and Ruff, M. (2013). *A forensic analysis and comparison of solid state drive data retention with trim enabled file systems* in proceedings of the 11th australian digital forensics conference. sri security research institute, edith cowan university, perth, western australia.

Olson, A. and Langlois, D. (2008). *Solid State Drives Data Reliability and Lifetime.*

Samsung, C. (2016). *SSD 850 EVO Overview* `http://www.samsung.com/global/business/semiconductor/minisite/SSD/global/html/ssd850evo/overview.html`.

Slivka, E. (2011). *Mac OS X 10.6.8 Brings TRIM Support for Apple SSDs, Graphics Improvements [Updated]* `http://www.macrumors.com/2011/06/27/mac-os-x-10-6-8-brings-trim-support-for-apple-ssds-graphics-improvements/`.

 ©2016 SBC — Soc. Bras. de Computação